
Proyecto 2 – Introducción a la Programación y Computación 2

202308221 – Jose Juan Laynez Zapeta

Resumen

El siguiente documento presenta información relevante acerca del proyecto 2 del curso de introducción a la programación 2.

Con este proyecto, se pretende aprender diversos temas de programación como programación orientada a objetos, arquitectura de software, manipulación de archivos XML, creación de graficos en Graphviz, y como punto fuerte, el aprendizaje de datos abstractos no lineales.

El sistema busca la gestión de libros, de manera jerárquica, con categorías y subcategorías. Esto se logra por medio de estructuras de datos lineares y no lineares propios, no del lenguaje o del Framework. Se hizo uso principalmente del lenguaje C#, pero se trabajó también con HTML, CSS, Razor Pages, Graphviz con el Framework .NET y con ASP.NET.

Palabras clave

Programación, Tipos de Datos Abstractos, Clases, Objetos, Arboles, Libros.

Abstract

The following document presents relevant information regarding the second project of the Introduction to Programming 2 course.

This project aims to cover various programming topics such as object-oriented programming, software architecture, XML file manipulation, and graph creation using Graphviz—with a particular focus on learning about non-linear abstract data types.

The system looks to manage books, in a hierarchical manner, with categories and subcategories. This is achieved through own linear and non-linear data types, not through the ones from the language or Framework. It was made mainly with C#, but it was also worked with HTML, CSS, Razor Pages, Graphviz, the .NET Framework and with ASP.NET.

Keywords

Programming, Abstract Data types, classes, objects, Trees, Books.

Introducción

A continuación, se presentará mas información acerca del proyecto. Se trata de un sistema que solicitado por una cadena de librerías, que necesita administrar un catalogo de una cantidad grande de libros distribuidos en varias sucursales, y que actualmente pierde tiempo ya que los libros estan en lineas extensas y poco estructuradas.

Para resolver esta problemática, se hizo uso de TDAs propios, Programacion orientada a objetos, manipulacion de XML, y generacion de reportes con Graphviz. El sistema organiza el catalogo de la empresa de forma jerárquica, con categorías, subcategorías y libros, de manera similar a como una biblioteca fisica, con secciones y estanterías, y permite localizar cualquier libro segun su ISBN, sin que tengan que visitar a todos los libros.

El proyecto consiste en un catalogo administrable desde una interfaz web que permite la carga de información inicial desde archivos XML, registrar y eliminar libros, agregar categorías, consultar con el ISBN los datos de los libros, ver el mayor y menor, y visualizar la estructura del catalogo.

Se desarrolló en Visual Studio Code, usando ASP.NET Core Razor Pages para la interfaz web.

Desarrollo del tema

El proyecto fue elaborado con base en los siguientes aspectos fundamentales:

- a. Implementación de Tipos de Datos Abstractos (TDA):

Una de las partes más importantes del proyecto fue construir las propias estructuras de datos desde cero, sin apoyarse en las colecciones que ya trae C# (List, Stack, Queue, etc.). Para el índice de libros se

implementó un Árbol Binario de Búsqueda (ArbolLibros) ordenado por ISBN, lo que permite buscar, insertar y eliminar en tiempo logarítmico, y obtener el libro con el ISBN menor o mayor recorriendo el árbol hacia la izquierda o hacia la derecha respectivamente. Un recorrido in-orden del árbol entrega los libros ya ordenados de forma ascendente, sin necesidad de un paso adicional de ordenamiento.

Para la jerarquía de categorías se optó por una representación distinta a la de un árbol binario convencional, ya que una categoría puede tener cualquier número de subcategorías. Se usó la técnica de "primer hijo / siguiente hermano": cada nodo de categoría (NodoArbolCategoria) apunta únicamente a su primer hijo, y cada hijo apunta a su hermano siguiente, de modo que recorriendo esa cadena se visitan todos los hijos de un nodo usando solamente punteros binarios. Insertando cada hermano nuevo en su posición alfabética correcta dentro de esa cadena, el recorrido entrega las categorías ya ordenadas alfabéticamente. Como el catálogo puede tener varias categorías principales sin padre (en realidad, un bosque de árboles), se introdujo una raíz oculta que nunca se muestra al usuario, cuyos hijos son las categorías de nivel superior; así toda la jerarquía se administra como un único árbol.

Cada categoría además guarda su propio índice local de libros: una instancia independiente de ArbolLibros con los libros que pertenecen directamente a ella, sin contar los de sus subcategorías. Esto significa que un mismo objeto Libro puede estar referenciado simultáneamente por dos nodos distintos —uno en el índice global del catálogo, y otro en el índice local de su categoría— cada uno con su propia posición dentro de su respectivo árbol.

Finalmente, para transportar resultados entre las distintas capas del sistema (por ejemplo, el resultado de un recorrido en orden ascendente, o la estructura completa del catálogo) se implementaron listas enlazadas propias: `ListaEnlazadaLibros`, `ListaEnlazadaCategorias` y `ListaEnlazadaTexto` (esta última para acumular mensajes de error). En vez de recibir un delegado (`Action`) como parámetro para procesar cada elemento, cada recorrido recibe una de estas listas y la va llenando; quien la recibe después la recorre con un ciclo `while` manual desde su nodo cabeza. Esto evita cualquier dependencia de colecciones genéricas incluso para pasar datos entre capas..

b. Programación Orientada a Objetos:

El diseño se organizó en tres paquetes: Modelos (las entidades Libro y Categoría, clases de datos puras sin lógica de estructura), Structures (los TDA propios: nodos y árboles/listas), y Logica (las clases que coordinan el comportamiento del sistema). La clase `Catalogo` actúa como fachada: es la única clase que la interfaz web utiliza directamente, y por dentro mantiene sincronizados el índice global de libros y el árbol de categorías, de modo que registrar o eliminar un libro actualiza ambas estructuras de forma consistente. `CargadorXml` se encarga de leer los archivos de configuración y poblar el `Catalogo`, y `GeneradorGraphviz` construye el archivo `.dot` y produce la imagen que se muestra en la interfaz. El diagrama de clases completo se incluye en la sección de Anexos.

c. Manipulación de archivos XML

El catálogo puede inicializarse cargando uno o varios archivos de configuración en formato XML. Cada archivo puede contener una lista de categorías (cada

una con un nombre y, opcionalmente, el nombre de su categoría padre) y una lista de libros (ISBN, título, autor y categoría), y ambas listas son opcionales, ya que la carga es incremental: se pueden subir varios archivos, uno tras otro, y cada uno se suma al catálogo que ya se tenía cargado.

Un detalle importante es que el XML no garantiza que una categoría padre aparezca antes que su hijo dentro del archivo. Para resolverlo, `CargadorXml` intenta insertar cada categoría en una primera pasada; las que no se puedan insertar porque su padre todavía no existe se dejan pendientes en listas enlazadas propias, y se reintentan en pasadas siguientes hasta que ya no haya progreso. Cualquier error encontrado durante la carga (un ISBN repetido, una categoría inexistente, un valor numérico inválido) se acumula en un objeto `ResultadoCarga` en vez de detener el proceso completo, de modo que el resto del archivo se siga procesando y el usuario reciba al final un resumen de lo que se agregó y de lo que falló.

d. Algoritmos de búsqueda de caminos

Para cumplir con el requerimiento de mostrar gráficamente los libros de una categoría o subcategoría en orden ascendente de ISBN, se implementó `GeneradorGraphviz`. Esta clase recorre la `ListaEnlazadaLibros` que entrega el `Catalogo` (ya ordenada, gracias al recorrido in-orden del árbol) y construye un archivo `.dot` donde cada libro es un nodo etiquetado con su ISBN, título y autor, conectado en secuencia con el siguiente libro; el orden ascendente queda representado directamente en la dirección de las flechas del diagrama. Después se invoca el ejecutable `dot` como un proceso externo, indicándole que genere una imagen PNG a partir de ese archivo, y la imagen resultante se sirve desde un endpoint de

la aplicación web para poder mostrarla dentro de una etiqueta .

e. Dificultades encontradas:

El mayor reto del proyecto fue diseñar las estructuras de manera que un mismo libro pudiera pertenecer simultáneamente a dos índices distintos (el global y el de su categoría) sin duplicar la información ni perder la sincronía entre ambos al insertar o eliminar. Otro reto fue decidir cómo representar una jerarquía de categorías que en realidad podía tener varias raíces (varias categorías principales sin padre) usando una sola estructura de árbol, lo que llevó a introducir la raíz oculta. Por último, evitar por completo el uso de listas o delegados genéricos de C#, incluso para tareas aparentemente simples como devolver el resultado de un recorrido, obligó a diseñar desde el principio pequeñas listas enlazadas propias para cada tipo de dato que se necesitaba transportar entre capas.

Conclusiones

Después de completar este proyecto, puedo destacar varias conclusiones importantes:

1. Comprensión profunda de las estructuras de datos: al tener que construir los árboles y listas desde cero, se entendió con claridad cómo funcionan realmente por dentro: qué ocurre en memoria al insertar, buscar o eliminar un elemento, en vez de tratarlas como una "caja negra" que ya viene resuelta en el lenguaje.
2. Un mismo objeto puede vivir en varias estructuras a la vez: separar la entidad (Libro, Categoría) de los nodos que la envuelven en cada estructura (NodoArbolLibro, NodoLibro) permitió que un mismo libro participara simultáneamente en dos árboles distintos sin duplicar información.

3. Importancia de una fachada clara: centralizar toda la lógica del catálogo en una sola clase (Catalogo) simplificó enormemente la interfaz web, que nunca tuvo que manipular los árboles directamente ni preocuparse por mantenerlos sincronizados.
4. Manejo robusto de archivos de entrada: procesar un XML incremental, sin garantía de orden entre categorías padre e hijo, exigió pensar en una solución de varias pasadas y en reportar errores sin detener toda la carga.
5. Las herramientas externas amplían lo que un programa puede mostrar: usar Graphviz permitió generar un reporte visual del catálogo sin tener que implementar un motor de dibujo propio, aprovechando una herramienta ya pensada para este tipo de diagramas.
6. Valor del versionamiento: llevar el desarrollo en releases sucesivos en GitHub ayudó a mantener un historial claro de avance y a poder aislar en qué punto se introdujo cada cambio.

Referencias bibliográficas

Documentación oficial de ASP.NET Core. Microsoft.
Disponible en: <https://learn.microsoft.com/aspnet/core/>

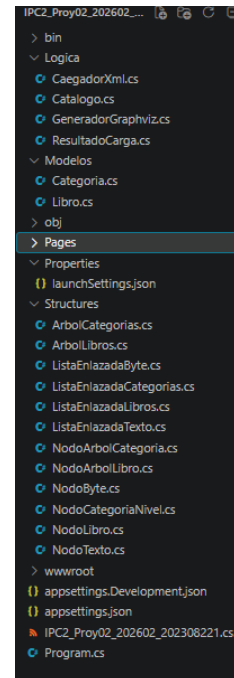
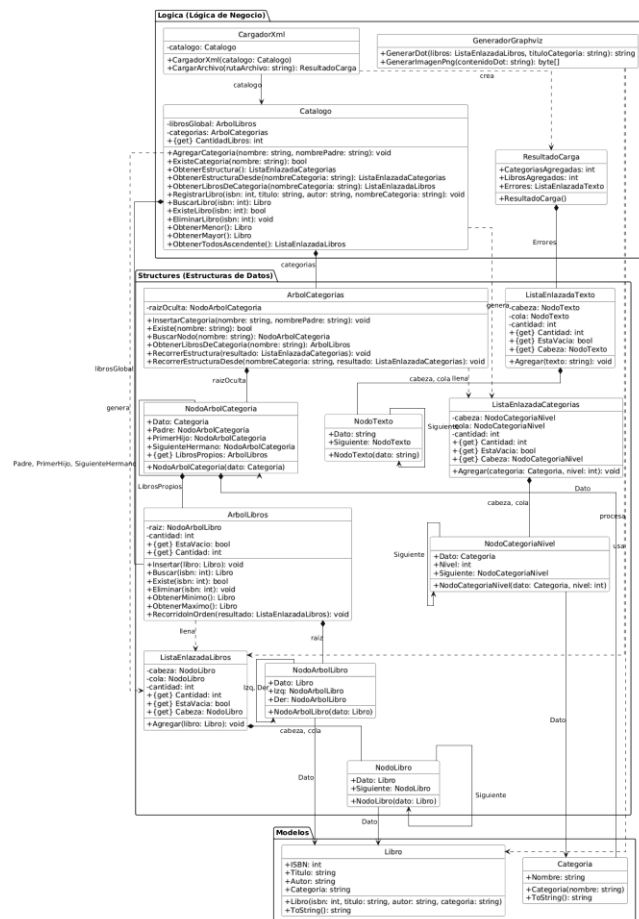
Joyanes Aguilar, L. (2008). Fundamentos de Programación: Algoritmos, Estructura de Datos y Objetos. McGraw-Hill Education.

Documentación de Graphviz. AT&T Research Labs.
Disponible en: <https://graphviz.org/documentation/>

Documentación oficial de C# y .NET. Microsoft.
Disponible en: <https://learn.microsoft.com/dotnet/csharp/>

Material de clase del curso Introducción a la Programación y Computación 2, USAC, 2do semestre 2026.

Extensión:



*Estructura del Proyecto dentro de Visual Studio Code –
Elaboracion propia en Visual Studio Code*

Biблиотека USAC

InicioLibrosCategoríasCarga XML

Gestión de categorías

Agregar categoría

Nombre

Categoría padre (opcional)

(Ninguna — categoría principal) ▾

Agregar

Estructura de categorías

Catalogo — [ver desde aquí](#) | [ver libros](#)
Arte — [ver desde aquí](#) | [ver libros](#)
 Fotografía — [ver desde aquí](#) | [ver libros](#)
 Tecnología — [ver desde aquí](#) | [ver libros](#)
 Tema 102 — [ver desde aquí](#) | [ver libros](#)
 Tema 123 — [ver desde aquí](#) | [ver libros](#)
 Tema 129 — [ver desde aquí](#) | [ver libros](#)
 Tema 139 — [ver desde aquí](#) | [ver libros](#)
 Tema 166 — [ver desde aquí](#) | [ver libros](#)
 Tema 174 — [ver desde aquí](#) | [ver libros](#)
 Tema 118 — [ver desde aquí](#) | [ver libros](#)
 Tema 58 — [ver desde aquí](#) | [ver libros](#)
 Tema 138 — [ver desde aquí](#) | [ver libros](#)

Diagrama de clases. Fuente: Elaboración propia mediante PlantUML.

Programa en funcionamiento – Elaboración Propia